DPA System Architecture

DPA (Digital Playback Asset) — brought to you by **The DPAC** (Digital Playback Asset Consortium)

Version: 1.0.0 | **Platform:** ESP32 + Angular 21

System Overview

DPA (Digital Playback Asset) is a physical music player brought to you by **The DPAC** (Digital Playback Asset Consortium). Each device holds one album in hi-fi FLAC 24/96 with music videos, downloadable capsule perks, and a mesh network for synchronized playback across devices.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              FAN'S PHONE / BROWSER                       │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                         ANGULAR 21 PORTAL                        │   │
│   │                                                                  │   │
│   │   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────────┐  │   │
│   │   │ Fan Home │   │  Audio   │   │ Capsules │   │   Settings   │  │   │
│   │   │ (Albums) │   │ Settings │   │  (Perks) │   │  (LED/Auth)  │  │   │
│   │   └──────────┘   └──────────┘   └──────────┘   └──────────────┘  │   │
│   │        │              │              │               │          │   │
│   │        └──────────────┴──────────────┴───────────────┴──────────┘   │
│   │   ┌──────────────────────────────────────────────────────────┐   │
│   │   │              DeviceConnectionService                     │   │
│   │   │           (BLE / WiFi / NFC orchestrator)                │   │
│   │   └──────────────┬──────────────┬──────────────┬──────────────┘   │
│   │                  │              │              │                  │
│   │   ┌──────────────┐   ┌──────────┐   ┌──────────────────┐        │
│   │   │ BLE Service  │   │WiFi Service│   │  NFC Service     │        │
│   │   │ Web BT API   │   │ HTTP fetch │   │  Web NFC API     │        │
│   │   └──────────────┘   └──────────┘   └──────────────────┘        │
│   └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
                    │              │              │
                    │ BLE GATT     │ HTTP REST    │ NDEF
                    │              │              │
┌──────────────────────────────────────────────────────────────────────────┐
│                              ESP32 DPA DEVICE                            │
│                                                                          │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐            │
│   │   BLE    │   │  WiFi AP │   │  PN532   │   │  A2DP    │            │
│   │  GATT    │   │  + REST  │   │   NFC    │   │  Source  │            │
│   └──────────┘   └──────────┘   └──────────┘   └──────────┘            │
│        │              │              │              │                   │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                        COMMAND DISPATCHER                        │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│      │          │          │          │          │                     │
│   ┌──────┐   ┌──────┐   ┌──────┐   ┌──────┐   ┌──────────┐            │
│   │Player│   │ LED  │   │ NVS  │   │ESP-NOW│   │  XTSD    │            │
│   │Engine│   │ RGB  │   │Config│   │ Mesh  │   │ Storage  │            │
│   └──────┘   └──────┘   └──────┘   └──────┘   └──────────┘            │
│                                        │                                │
│                                        │                                │
```

```
│                                  ┌──────┴──────┐                          │
│                                  │ Other DPA   │                          │
│                                  │  Devices    │                          │
│                                  └─────────────┘                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Connection Methods

NFC Tap (Instant Discovery)

Aspect	Detail
Hardware	PN532 via I2C (GPIO 21/22)
Tag Format	NDEF URL: `https://dpa.audio/d/{DUID}`
Portal API	Web NFC API (`NDEFReader.scan()`)
Flow	Phone tap -> read DUID -> portal auto-connects BLE
Browser Support	Android Chrome only

BLE (Persistent Control)

Aspect	Detail
Service D1A0	Info: DUID (D1A1), Status JSON (D1A2), FW Version (D1A3)
Service D1D0	Control: Command byte write (D1D1)
Portal API	Web Bluetooth API
Flow	Scan for service 0x D1A0 -> connect -> subscribe to D1A2 notify
Browser Support	Chrome, Edge, Opera (desktop + Android)

WiFi AP (Data Transfer)

Aspect	Detail
SSID	Device DUID (e.g., `DPA-AB12`)
Password	`dpa12345`
IP	`192.168.4.1`
Flow	Join device WiFi -> portal probes `/api/status` -> HTTP mode
Use Case	File uploads, theme push, capsule delivery

A2DP Source (Wireless Audio)

Aspect	Detail
Role	ESP32 acts as A2DP **Source** (sends audio)
Target	Bluetooth speakers, headphones

Codec	SBC (standard Bluetooth audio)
Control	Scan, connect, disconnect via BLE commands or WiFi API

Firmware REST API

All endpoints served on `http://192.168.4.1` when WiFi portal is active.

Status & Catalog

Method	Route	Response	Description
GET	`/api/status`	Status JSON	Device state, audio, battery, storage, mesh
GET	`/api/fan.json`	Catalog JSON	Song list, playlist, credits, videos
GET	`/api/creator.json`	Creator JSON	Features list + embedded status

Playback Control

Method	Route	Params	Description
GET	`/api/cmd`	`op=N`	Execute command by hex code (see Command Table)
GET	`/api/track`	`i=N`	Select track by index

Audio Settings

Method	Route	Params	Description
GET	`/api/volume`	`level=0-100`	Set volume (persisted to NVS)
GET	`/api/eq`	`preset=flat|bass|vocal|warm`	Set EQ preset
GET	`/api/mode`	`mode=normal|repeat_all|repeat_one|shuffle`	Set playback mode

Bluetooth A2DP

Method	Route	Params	Description
GET	`/api/a2dp/scan`	—	Start discovery, return found devices
GET	`/api/a2dp/devices`	—	List discovered devices
GET	`/api/a2dp/connect`	`addr=XX:XX:XX:XX:XX:XX`	Connect to A2DP sink
GET	`/api/a2dp/disconnect`	—	Disconnect current A2DP device

Storage

Method	Route	Params	Description
GET	`/api/storage`	—	XTSD flash usage (total, used, free, counts)
GET	`/api/tracks`	—	List files in /tracks/ directory

Content Push (Creator)

Method	Route	Body	Description
POST	`/api/theme`	Theme JSON	Push LED theme, stored in NVS
POST	`/api/capsule`	Capsule JSON	Trigger notification, broadcast to mesh
POST	`/api/manifest`	Manifest JSON	Update album metadata
POST	`/api/upload`	Multipart file	Upload .dpa encrypted file to XTSD flash

Mesh

Method	Route	Response	Description
GET	`/api/mesh`	Peer list JSON	ESP-NOW peer info (DUID, MAC, age)

BLE GATT Profile

Service: Device Info (UUID: 0000D1A0-...)

Characteristic	UUID	Properties	Format	Description
Device ID	D1A1	Read	UTF-8 string	DUID (e.g., "DPA-AB12")
Status	D1A2	Read, Notify	UTF-8 JSON	Full device status (same as /api/status)
FW Version	D1A3	Read	UTF-8 string	"1.0.0"

Service: Control (UUID: 0000D1D0-...)

Characteristic	UUID	Properties	Format	Description
Command	D1D1	Write	1 byte uint8	Command code (see table below)

Command Codes

Command	Hex	Dec	Description
Play	0x01	1	Start playback
Pause	0x02	2	Pause playback
Next Track	0x03	3	Skip to next track
Previous Track	0x04	4	Go to previous track
Get Status	0x10	16	Trigger BLE status notification
BT Pairing	0x20	32	Clear bonds, restart advertising
Start Portal	0x30	48	Enable WiFi AP + web server
Stop Portal	0x31	49	Disable WiFi AP
ESP-NOW Sync	0x40	64	Broadcast sync to mesh peers

ESP-NOW Discover	0x41	65	Broadcast mesh discovery
A2DP Scan	0x50	80	Start BT Classic device scan
A2DP Connect	0x51	81	Connect to last scanned device
A2DP Disconnect	0x52	82	Disconnect A2DP sink
Volume Up	0x60	96	Increase volume by 5%
Volume Down	0x61	97	Decrease volume by 5%
Cycle Mode	0x62	98	Cycle playback mode
Cycle EQ	0x63	99	Cycle EQ preset

.dpa Encrypted Container Format

All content files (FLAC audio, videos, capsules) are encrypted into `.dpa` containers that can only be decrypted by the target device.

File Structure

```
Offset  Size    Field               Description
──────  ──────  ──────────────────  ──────────────────────────────
0x00    4       Magic               "DPA\x01" (0x44 0x50 0x41 0x01)
0x04    1       Version             0x01
0x05    1       Flags               bit0=FLAC, bit1=Video, bit2=Capsule
0x06    32      DUID Hash           SHA-256(DUID + "DPA-MASTER-2026")
0x26    12      IV / Nonce          Random 12-byte nonce
0x32    N       Encrypted Payload   AES-256-GCM ciphertext + 16-byte tag
```

Total header size: 50 bytes

Encryption Flow

```
                    Portal (Browser)                    Device (ESP32)
                    ────────────────                    ──────────────

1. Creator uploads   ┌─────────────┐
   raw FLAC file     │  Raw Audio  │
                     └─────────────┘
                           │
                           │
2. Derive key       SHA-256(DUID + "DPA-MASTER-2026")
                           │
                           │
                     ┌─────────────┐
3. Encrypt           │ AES-256-GCM │
   (WebCrypto)       │ + random IV │
                     └─────────────┘
                           │
                           │
4. Build .dpa        ┌─────────────┐
   container         │  .dpa File  │
                     └─────────────┘
                           │
                           │
5. Transfer         ─────── POST /api/upload ──────►  ┌─────────────┐
   via WiFi                                           │  XTSD Flash │
                                                      │  /tracks/   │
```

```
6. Playback
   on device
```

```
└──────────────┘
       │
       ┌
┌──────────────┐
│  Verify DUID │
│  Hash Match  │
└──────────────┘
       │
       ┌
┌──────────────┐
│  AES-256-GCM │
│  Decrypt     │
│  (mbedtls)   │
└──────────────┘
       │
       ┌
┌──────────────┐
│  FLAC Decode │
│  (dr_flac)   │
└──────────────┘
       │
       ┌
┌──────────────┐
│  I2S Output  │
│  PCM5122 DAC │
└──────────────┘
```

Key Derivation

Both the portal (browser WebCrypto) and firmware (mbedtls) derive the same key:

```
key_material = UTF8_ENCODE(DUID + "DPA-MASTER-2026")
key          = SHA-256(key_material)              // 32 bytes = AES-256
```

DUID Hash Verification

Before decryption, the device checks that the file was encrypted for this device:

```
stored_hash   = bytes[6..38] from .dpa header
expected_hash = SHA-256(own_DUID + "DPA-MASTER-2026")

if stored_hash != expected_hash:
    reject with DPA_WRONG_DEVICE
```

ESP-NOW Mesh Protocol

DPA devices form an ad-hoc mesh network for synchronized playback and capsule notification relay.

Message Types

Type	Code	Struct	Size	Purpose
Discovery	0x01	ENDiscovery	28 bytes	Announce presence
Discovery ACK	0x02	ENDiscovery	28 bytes	Respond to discovery
Command	0x03	ENCommand	30 bytes	Remote playback control
Status	0x04	ENStatus	22 bytes	Periodic state broadcast

Sync	0x05	ENSync	25 bytes	Synchronized playback start
Capsule	0x06	ENCapsule	46 bytes	Capsule notification relay

Packet Header (all messages)

```
Offset  Size  Field        Description
0       1     msgType      Message type code
1       1     seqNum       Sequence number (0-255, wrapping)
2       12    senderDuid   Sender device DUID string
14      2     payloadLen   Payload size after header
```

Mesh Behavior

- Discovery broadcasts every 5 seconds
- Peers timeout after 15 seconds of inactivity
- Maximum 6 simultaneous peers
- Channel 6 (matches WiFi AP channel)
- No encryption (local proximity mesh)
- Capsule notifications relay through mesh so all nearby devices flash

---

Status JSON Schema

Returned by `GET /api/status` and BLE characteristic D1A2:

```json
{
  "name": "dpa-device",
  "ver": "1.0.0",
  "env": "dev",
  "duid": "DPA-AB12",
  "ble": true,
  "wifi": true,
  "ip": "192.168.4.1",
  "uptime_s": 3600,

  "audio": {
    "volume": 75,
    "eq": "flat",
    "mode": "normal",
    "a2dp": "disconnected",
    "a2dpDevice": ""
  },

  "battery": {
    "percent": 85,
    "voltage": 3.95,
    "charging": false
  },

  "storage": {
    "totalMB": 29000,
    "usedMB": 4200,
    "freeMB": 24800,
```

```
    "trackCount": 12,
    "capsuleCount": 3,
    "videoCount": 2
  },

  "espnow": {
    "active": true,
    "peers": 2,
    "peerList": [
      { "duid": "DPA-CD34", "age": 3 },
      { "duid": "DPA-EF56", "age": 8 }
    ]
  },

  "player": {
    "trackIndex": 0,
    "trackId": "s1",
    "playing": true,
    "posMs": 45000
  },

  "counts": {
    "play": 15,
    "pause": 8,
    "next": 12,
    "prev": 3
  }
}
```

LED Notification System

Capsule Perk Types

Each perk type has a configurable color (set in theme editor) and a fixed animation pattern:

Perk Type	Default Color	Animation	Duration
Concert	#ff4bcb (Magenta)	Rapid pulse 3x, then breathing	~33s
Video	#00f1df (Cyan)	Slow fade in/out, repeat 2x	~14s
Merch	#ffcc33 (Gold)	3 quick flashes, solid glow	~11s
Remix	#ff4500 (Hot Orange)	Rhythmic pulse at 120 BPM	15s
Signing	#7d29ff (Purple)	Slow breathing (inhale/exhale)	30s
Other	#ffffff (White)	2 gentle flashes	~1.2s

LED State Modes

Mode	Color Source	Animation	When
Idle	Theme idle color	Breathing	Device on, not playing

Playback	Theme playback color	Pulse	Currently playing audio
Charging	Theme charging color	Slow breathing	Battery charging
Notification	DCNP perk color	Per-type pattern	Capsule received

Theme NVS Storage

LED theme settings persist across reboots:

NVS Key	Type	Default	Description
`led_idle`	String	`#00ff88`	Idle state LED color
`led_play`	String	`#00aaff`	Playback state LED color
`led_charge`	String	`#ffaa00`	Charging state LED color
`led_idle_pat`	String	`breathing`	Idle animation pattern
`dcnp_concert`	String	`#ff4bcb`	Concert notification color
`dcnp_video`	String	`#00f1df`	Video notification color
`dcnp_merch`	String	`#ffcc33`	Merch notification color
`dcnp_remix`	String	`#ff4500`	Remix notification color
`dcnp_signing`	String	`#7d29ff`	Signing notification color
`dcnp_other`	String	`#ffffff`	Other notification color

Portal Architecture

Angular 21 Application

Technology	Version	Purpose
Angular	21	Application framework
Vite	6.x	Build tool
Tailwind CSS	4.x	Utility-first styling
TypeScript	5.7	Type safety

Key Services

Service	File	Responsibility
DeviceConnectionService	`device-connection.service.ts`	Connection state machine (BLE/WiFi/NFC)
DeviceBleService	`device-ble.service.ts`	Web Bluetooth GATT operations
DeviceWifiService	`device-wifi.service.ts`	HTTP client for device REST API
CryptoService	`crypto.service.ts`	.dpa encryption/decryption (WebCrypto)

PlayerService	`player.service.ts`	Playback with device command routing
DataService	`data.service.ts`	Album/track data management
CartService	`cart.service.ts`	Shopping cart for capsule marketplace

Route Structure

```
/login                          Login screen
/artist/                        Creator portal
  /dashboard                    Album dashboard
  /albums/new                   Create new album
  /albums/:id/                  Album editor
    /overview                   Album overview
    /metadata                   Metadata editor
    /tracks                     Track list + upload
    /booklet                    Digital booklet
    /theme                      LED theme editor
    /perks                      Capsule perks console
    /pricing                    Pricing configuration
    /devices                    Device management
  /account                      Account settings
  /fleet                        Fleet tracker (mesh map)
/fan/                           Fan portal
  /auth                         Fan authentication
  /app/                         Fan app (layout + player)
    /home                       Album catalog
    /album/:id                  Album detail + player
    /capsules                   Capsule inbox
    /marketplace                Capsule marketplace
    /devices                    Device registration
    /checkout                   Cart checkout
    /audio                      Audio settings (volume/EQ/A2DP)
    /settings                   Profile + LED theme
```

NVS (Non-Volatile Storage) Keys

Key	Type	Default	Description
`duid`	String	Auto-generated	Device unique ID (DPA-XXXX)
`volume`	UChar	75	Audio volume (0-100)
`eq`	UChar	0	EQ preset index
`mode`	UChar	0	Playback mode index
`led_idle`	String	#00ff88	Idle LED color
`led_play`	String	#00aaff	Playback LED color
`led_charge`	String	#ffaa00	Charging LED color
`led_idle_pat`	String	breathing	Idle animation

`dcnp_*`	String	Per-type	6 notification colors
`theme`	String	{}	Full theme JSON blob
`manifest`	String	{}	Album manifest JSON

Build & Compile Flags

Firmware Feature Matrix

Flag	Controls	Dependencies
`HAS_DAC`	PCM5122 + I2S + XTSD flash + FLAC decode	dr_flac.h, mbedtls
`HAS_NFC`	PN532 NFC tag emulation	Adafruit_PN532 library
`HAS_A2DP`	Bluetooth A2DP Source (auto-detected)	Arduino BT config = "BT + BLE"

Build Combinations

Configuration	Use Case	Flash Size
No flags	Simulation / development	~1.2 MB
HAS_DAC	Real audio without NFC	~1.5 MB
HAS_DAC + HAS_NFC	Full hardware build	~1.6 MB
HAS_DAC + HAS_NFC + HAS_A2DP	Complete with wireless audio	~1.8 MB

Portal Build

```
npm install          # Install dependencies
npm run dev          # Development server (localhost:4200)
npx ng build        # Production build (output: dist/)
```

Zero external API keys required. No Google or AI service dependencies.